

Nagy házi feladat dokumentáció

The Last Stand

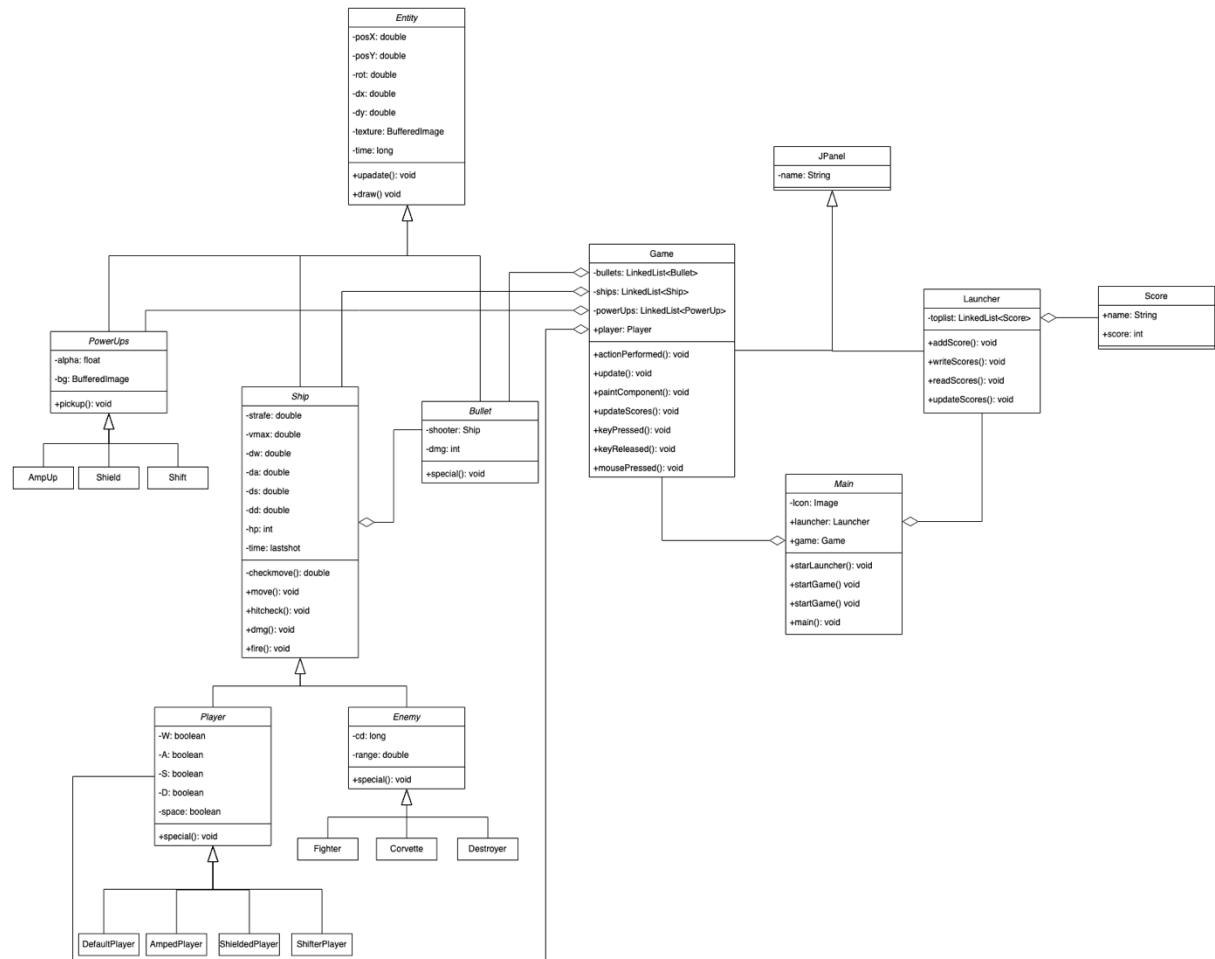
Borbényi Dániel (H0XAD8) – Programozás Alapjai 3

Tartalomjegyzék:

Osztálydiagram	3
Package main	4
Main	4
Launcher	4
Game	5
Score	7
Package entities	8
Entity	8
Package entities.bullet	9
Bullet	9
Package entities.ships	10
Ship	10
Package entities.ships.enemies	11
Enemy	11
Fighter	11
Corvette	11
Destroyer	12
Package entities.ships.player	13
Player	13
DefaultPlayer	13
AmpedPlayer	13
ShieldedPlayer	14
ShifterPlayer	14
Package entities.upgrades	16
PowerUp	16
Ampup	16
Shield	16
Felhasználói Útmutató	18
Launcher	18

Játék:	18
Játékos:	18
Ellenfelek:.....	18

Osztálydiagram



Package main

Ez a package a program futásához legfontosabb osztályokat tartalmazza.

Main

```
/**
 *
 * A program indulása
 */
public class Main

/**
 * Inicializálja az ablakot amiben a launcher van, és launchert
 */
public static void startLauncher()

/**
 * inicializálja az ablakot amiben a játék van és a játékot
 */
public static void startGame() throws UnsupportedOperationException,
LineUnavailableException, IOException
```

Launcher

```
/**
 * Ez kis ablak a program indítója, ami a toplistát is kezeli.
 */
public class Launcher extends JPanel

/**
 * Egy szerű szöveg, ami csak annyit ír: Enter your name
 */
private final JLabel nametext = new JLabel();

/**
 * Ide kell beírni a nevet
 */
private final JTextField name = new JTextField(5);

/**
 * Játék indítása gomb
 */
private final JButton launch = new JButton("PLAY");

/**
 * A hang ki/be kapcsoló gomb
 */
private final JToggleButton sound = new JToggleButton("SOUND ON");

/**
 * A kilépés gomb
 */
private final JButton exit = new JButton("EXIT");

/**
 * A toplistát kiíró szöveg
 */
private final JTextArea scores = new JTextArea();

/**
 * A legjobb 10 Scoret tartalmazó lista
 */
protected final LinkedList<Score> toplist = new LinkedList<>();
```

```

/**
 * A játékindító gomb Listenerje
 */
private static class LaunchListener implements ActionListener

/**
 * A hang kapcsoló Listenerje
 */
private class SoundListener implements ActionListener

/**
 * A kilépő gomb Listenerje
 */
private static class ExitListener implements ActionListener

/**
 * A beírt név és pontszámát menti a pontszámok listába
 * @param n A pontszám
 */
public void addScore(int n)

/**
 * Kiírja a toplist.txt-be a toplistát
 */
public void writescores()

/**
 * Beolvassa a toplist.txt fileból a toplistát
 */
public void readscores()

/**
 * A toplista kiírását frissíti
 */
public void updateScores()

```

Game

```

**
 * A játék osztálya. Egyszerre csak egyszer inicializálható
 */
public class Game extends JPanel implements ActionListener, KeyListener ,
MouseListener

/**
 * A játék ablaka
 */
private final JFrame window;

/**
 * Alap paraméterek, felbontás és tickrate
 */
private final int resX, resY, Hz;

/**
 * Az utolsó tick ideje
 */
private long t_last;

/**
 * A játék indítási ideje

```

```

    */
private final long t_start;

/**
 * Az épp létező hajók listája
 */
protected final LinkedList<Ship> ships = new LinkedList<>();

/**
 * Az épp létező töltények listája
 */
protected final LinkedList<Bullet> bullets = new LinkedList<>();

/**
 * Az épp pályán levő fejlesztések listája
 */
protected final LinkedList<PowerUp> powerUps = new LinkedList<>();

/**
 * A háttérkép, azért van külön kezelve mert ez nem lehet BufferedImage
 */
private final Image background;

/**
 * A pontszám
 */
private int score = 0;

/**
 * Igaz, ha átváltottunk hardmode-ba
 */
private boolean hardmode = false;

/**
 * Igaz, ha meghaltunk
 */
protected boolean over = false;

/**
 * A játéko objektuma
 * Ez íródik felül ha egy fejlesztést felszedünk
 */
public Player player;

/**
 * A zene beolvasását kezelő Stream
 */
public AudioInputStream stream;

/**
 * A zene lejátszását kezelő objektum
 */
public Clip clip;

/**
 * A zene hangerőkezeléséhez szükséges objektum
 */
public FloatControl fc;

/**
 * A játék textúráit tartalmazó map

```

```

    */
    public HashMap<String, BufferedImage> textures = new HashMap<>();

    /**
     * A pontos tick frissítés hívásra használt osztály
     */
    public Timer timer;

    /**
     * Ez a függvény hívódik meg a Timer által 16 ms-ként.
     * Ez a függvény hívja meg az update() és repaint függvényt()
     * Valamit a hardmode-ba váltást is kezeli
     * @param e the event to be processed
     */
    @Override
    public void actionPerformed(ActionEvent e)

    /**
     * A minden tickenként lefutó, mindent frissítő függvény.
     * Ugyanitt történik az új ellenfelek és fejlesztések hozzáadása.
     */
    private void update()

    /**
     * Kirajzolja a képernyőre az összes komponenst
     * @param g A JPanel Graphicse
     */
    @Override
    public void paintComponent(Graphics g)

    /**
     * A billentyűk lenyomásával foglalkozó függvény
     * @param e A lenyomott billentyű adatai
     */
    @Override
    public void keyPressed(KeyEvent e)

    /**
     * A felengedett billentyűket figyeli.
     * Ebben a függvényben záródik be az ablak az
     * Esc gomb megnyomásával miután meghaltunk.
     * @param e the event to be processed
     */
    @Override
    public void keyReleased(KeyEvent e)

    /**
     * Ha egy egérgomb lenyomódik, ez meghívódik
     * @param e Az egér adatai (pozíció, lenyomott gomb)
     */
    @Override
    public void mousePressed(MouseEvent e)

```

Score

```

    /**
     * Ez a class a szerializálást létrehozó adattípus, ami a ranglistához
     kell
     */
    public class Score implements Serializable

```

Package entities

Entity

```
/**
 * A játék absztrakt felépítőköve
 * A pozíciót, forgatását, és a pillanatnyi mozgulást tárolja
 * Valamint a kirajzolandó képet
 * A játékban minden kirajzolt elem egy entitás
 */
public abstract class Entity

/**
 * posX: 0-1 közötti érték, 0 a kép bal oldala és 1 a kép jobb oldala
 * posY: 0-1/y*x közötti érték, 0 a kép teteje és x/y a kép alja
 * rot: 0-2 a fordulata megadva radiánban
 * dx: pillanatnyi mozgulás az x tengelyen
 * dy: pillanatnyi mozgulás az y tengelyen
 */
private double posX, posY, rot, dx, dy;

/**
 * A lövedékeknel használt konstruktor
 * Forgás helyett a cél van megadva
 * @param x0 kezdeti x
 * @param y0 kezdeti y
 * @param x cél x
 * @param y cél y
 */
public Entity(double x0, double y0, double x, double y)

/**
 * A minden tickben meghívott update() függvény a mozgási logikát
 tartalmazza
 */
abstract public void update();

/**
 * A minden tickben meghívott kirajzoló függvény
 * @param g A JPanel Graphics-e
 * @param o A JPanel maga
 */
public void draw(Graphics g, ImageObserver o)
```


Package entities.bullet

Bullet

```
/**
 * A lövedékek osztálya
 * Elmenti hogy mennyit sebet és ki lőtte
 * Önmagát nem tudja megsebezni a lövő, de a csapattársait igen
 */
public class Bullet extends Entity

/**
 * Az adott irányba arréblépteti
 */
@Override
public void update()

/**
 * A lövedéknek nincs textúrája csak egy fehér kör
 * @param g A JPanel Graphics-e
 * @param o A JPanel maga
 */
@Override
public void draw(Graphics g, ImageObserver o)
```

Package entities.ships

Ship

```
/**
 * A hajók absztrakt osztálya
 * A surlódási tényezőt tárolják és a maximális sebességet
 * Valamint a életpontot és mikor ltótt utoljára
 */
public abstract class Ship extends Entity
double strafe, vmax;

/**
 * Mind a 4 irányt külön kezeljük, igaz hogy 2-2 egymás ellentétje
 * Ez a surlódás implementációnak követelménye.
 */
double dw = 0, da = 0, ds = 0, dd = 0;

/**
 * 1 irányban ellenőrzi, hogy gyorsulnia kéne-e arra, vagy lassulnia
 * @param v A sebesség
 * @param key ha igaz gyorsul, ha hamis lassul
 * @return az új sebesség abban az irányban
 */
protected double checkmove(double v, boolean key)

/**
 * Tickenként hívódik
 * Megnézi mind a 4 irányban, hogy merre kéne mennie
 */
protected void move()

/**
 * Ellenőrzi hogy eltalálta-e egy lövedék
 * @param bullet A tesztelendő lövedék
 * @return Igaz ha talált
 */
public boolean hitcheck(Bullet bullet)

/**
 * Lövünk az egér irányába
 * @param x az egér x koordinátája
 * @param y az egér y koordinátája
 */
public void fire(double x, double y)

/**
 * Ha talált, veszítsen életpontot
 * @param dmg Veszítendő életpont
 */
public void damage(int dmg)
```

Package entities.ships.enemies

Enemy

```
/**
 * Az ellenfeleket általánosító absztrakt osztály
 * Tartalmazza, hogy milyen gyakran lőhet
 * És hogy milyen távolságra szeretne lenni a játékostól
 * Az ellenfelek arra néznek amerre a játékos van (kivéve a Fighter)
 */
public abstract class Enemy extends Ship

/**
 * Tickenként meghívódó frissítés, az új pozíciót megkeresi
 * Lő ha tud
 */
@Override
public void update()

/**
 * Kilő egy lövedéket a játékos irányába
 * @param x A játékos x koordinátája
 * @param y A játékos y koordinátája
 */
public void fire(double x, double y)
```

Fighter

```
/**
 * A legkisebb ellenfél, lőni nem tud, cserébe a játékost zavarja,
 * blokkolja lövedékeit
 */
public class Fighter extends Enemy

/**
 * Ez a hajó arra néz, amerre mozog, és nem amerre a játékos van
 */
@Override
public void update()

/**
 * Ez a hajó nem tud lőni, ez a fv nem csinál semmit
 * @param x A játékos x koordinátája
 * @param y A játékos y koordinátája
 */
@Override
public void fire(double x, double y){}

/**
 * Ennek a halyónak megnöveljük a hitboxát, mert nehéz eltalálni
 * @param bullet A tesztelendő lövedék
 * @return Igaz ha eltaláltuk
 */
@Override
public boolean hitcheck(Bullet bullet)
```

Corvette

```
/**
 * A közepes méretű ellenfél
 * Csak konstruktort tartalmazza a tökéletes belállításokkal
 */
public class Corvette extends Enemy
```

Destroyer

```
/**  
 * A nagy méretű ellenfél  
 * Csak konstruktort tartalmazza a tökéletes belállításokkal  
 */  
public class Destroyer extends Enemy
```

Package entities.ships.player

Player

```
/**
 * A játékost általánosító absztrakt osztály
 * Az épp lenyomott billentyűket tartja még számon
 */
public abstract class Player extends Ship

/**
 * A másoló konstruktor, ami egy fejlesztés felvételekor hívódik meg
 * @param override az új textúra
 */
public Player(BufferedImage override)

/**
 * Lellenőrzi a lenyomott billentyűket
 */
@Override
public void update()

/**
 * Ha meg lettünk sebezve, meghalunk és GAMEOVER
 * @param dmg Veszítendő életpont
 */
@Override
public void damage(int dmg)
    throw new RuntimeException("GAME OVER");
}

/**
 * Ez a függvény a jobb klick kezelésért felel
 */
public void special(){}

/**
 * Az adott irányba teleportál minket egy kicsit
 */
public void pressedSpace()
    if (W)
        setPosY(getPosY() - 0.06);
    if (A)
        setPosX(getPosX() - 0.06);
    if (S)
        setPosY(getPosY() + 0.06);
    if (D)
        setPosX(getPosX() + 0.06);
}
```

DefaultPlayer

```
/**
 * Az átlagos játékos, mindenben a legalapabb
 */
public class DefaultPlayer extends Player
```

AmpedPlayer

```
/**
 * Az erőben fejlesztett játékos
 * Megnövelt tűzerő
```

```

    */
public class AmpedPlayer extends Player

/**
 * A jákos lövedékei megölnek mindenkit egy találattal
 * @param x az egér x koordinátája
 * @param y az egér y koordinátája
 */
@Override
public void fire(double x, double y)

```

ShieldedPlayer

```

/**
 * A védelemben fejlesztett játékos
 * Van egy pajzsa ami levéd 1 lövést
 */
public class ShieldedPlayer extends Player

/**
 * Jobbklikre a pajzsot ki/be állítjuk
 */
@Override
public void special()

/**
 * ha pajzsunk fent van, elpbb az tűnik el nem mi
 * @param bullet A tesztelendő lövedék
 * @return igaz ha eltaláltak minket
 */
@Override
public boolean hitcheck(Bullet bullet)

/**
 * Ha pajzsunk fent van, azt is rajzoljuk ki
 * @param g A JPanel Graphics-e
 * @param o A JPanel maga
 */
@Override
public void draw(Graphics g, ImageObserver o)

```

ShifterPlayer

```

/**
 * Az instabil játékos
 * Atvált egy másik dimenzióban ahol nem találhatják el
 * De nem lóhet
 * Instabilitás miatt bármikor visszakerülhet a játéktérbe
 */
public class ShifterPlayer extends Player

/**
 * Ha shiftelünk lassabban mozgunk és vagy egy kicsi esélyünk visszakerülni
 */
@Override
public void update()

/**
 * Ha shiftelve vagyunk, nem találhatnak el minket
 * @param bullet A tesztelendő lövedék
 * @return
 */

```

```
@Override
public boolean hitcheck(Bullet bullet)

/**
 * Ha shiftelve vagyunk nem tüzelünk
 * @param x az egér x koordinátája
 * @param y az egér y koordinátája
 */
@Override
public void fire(double x, double y)

/**
 * Jobbklickre állítjuk a shift kapcsolót
 */
@Override
public void special()
    shifted = !shifted;
}

/**
 * Ha shiftel, halványan rajzolja ki
 * @param g A JPanel Graphics-e
 * @param o A JPanel maga
 */
@Override
public void draw(Graphics g, ImageObserver o)
```

Package entities.upgrades

PowerUp

```
/**
 * A fejlesztéseket általánosító absztrakt osztály
 * A kirajzolás logikáját tartalmazza és a felvételi feltételt
 */
public abstract class PowerUp extends Entity
/**
 * A háttérben lévő particle effect
 */
BufferedImage bg;
/**
 * A pillanatnyi átettszőség
 * Folyamatosan csökken
 * 1: teljesen látszik
 * 0: láthatalan
 */
private float alpha;

/**
 * Minden egyes fejlesztés létrehoz egy új hajót
 * @return Az új fejlesztett Player objektum
 */
public abstract Player pickup();

/**
 * Növeljük az átettszőséget
 */
@Override
public void update()

/**
 * Elég közel van-e a játékos a fejlesztéshez?
 * @return igaz ha felvettük
 */
public boolean hitcheck()
```

Ampup

```
/**
 * A lőerő fejlesztés
 */
public class AmpUp extends PowerUp

/**
 * A játékosból AmpedUp lesz
 * @return Az új AmpedPlayer
 */
@Override
public Player pickup()
```

Shield

```
/**
 * Megnövekedett védelem fejlesztés
 */
public class Shield extends PowerUp

/**
 * A játékosból Shieleded lesz
```



```
    * @return Az új ShieldedPlayer
    */
@Override
public Player pickup()
/**
 * Kockázatos játék fejlesztése az alternatív
 * dimenzióba lépéssel
 */
public class Shift extends PowerUp
/**
 * A játékosból Shifter lesz
 * @return Az új ShifterPlayer
 */
@Override
public Player pickup()
```

Felhasználói Útmutató

Launcher:

A program elindításakor a Launcher fogad minket. Itt tudjuk beállítani a nevünket, lenémítani a játékot, és elindítani egy menetet belőle. A toplistát is itt találhatjuk, ami nem módosítható, és csak a legjobb 10 pontszámot és annak elérőjének nevét írja ki.

Játék:

A játék 2 fázissal rendelkezik, egy bemelegítéssel, amíg nem jönnek velünk szembe nagy hajók, és amikor már igen. A fázisváltásnál hirtelen nagyon megnövekedett számú ellenfelekre kell számítani.

Játékos:

A játékos különbözik a többi hajótól. Sokkal gyorsabb, és rezponzívabb. W A S D gombokkal lehet irányítani fel jobbra le és balra. A szóköz gombbal egy kicsi teleportálást idézhetünk elő. Az egér egy célkereszt, a bal klikkel tudunk tüzelni egyet. Ha eltalálunk egy ellenfelet, az veszít az életerejéből. A játékos eleinte egyszerű, de a játék során fejlesztéseket tud megszerezni, egyszerre 1-et tud magánál tartani. A jobbklikk hatására a fejlesztett játékosok speciális képességünket tudjuk meghívni. A fejlesztett játékosok:

- Kék: Pajzs: Jobbklikkel felhúzható egy pajzs, amit egy találat után újra fel kell húzni.
- Sárga: Lövéserő: A lövedékeik lézerré alakulnak, nagyon gyorsak, és azonnal megölnek minden ellenfelet
- Lila: Váltás: Jobbklikkel a játékos hajója egy alternatív dimenzióba kerül, ahol nem találhatják el, de nem is tud lőni. Ám ez az állapot instabil, bármikor visszakerülhetünk a valóságba.

Ellenfelek:

3 ellenfél jön ellenünk:

- Fighterek: ezek nem tudnak lőni ránk, és csak akadályt képeznek, nem tudunk lőni kifelé, bár a többi ellenfél hajók lelőhetik őket
- Corvettek: közepes méretű hajók. Ezek 4 másodpercenként tüzelnek ránk, és 3 életponttal rendelkeznek.
- Destroyerek: nagy méretű ellenséges hajók, csak a 2. fázisban találkozhatunk velük. Gyorsabban tüzelnek felénk és 5 életpontjuk van. Nehéz velük bánni!